

Link reading is unavailable in Expert Mode. Please use Instant Mode.

Replace Method with Method Object

تعریف ساده

هم همیشه درستش کرد، کل متد رو Extract Method وقتی به متد اینقدر طولانی و پیچیده‌ست که با بیر به یه کلاس جداگانه. متغیرهای محلی متد میشن فیلدهای کلاس، و بعد میتونی راحت‌تر متد رو به بخش‌های کوچیک بشکنی.

کی به این تکنیک نیاز داریم؟

شرایط	توضیح
Extract Method جواب نمیده	متغیرهای محلی زیادی وجود دارن که نمی‌دارن متد رو بشکنی
متد خیلی طولانیه	یه متد ۲۰۰-۳۰۰ خطی با کلی متغیر درهم
وابستگی داخلی زیاده	بخش‌های مختلف متد به هم گره خوردن

مثال قبل از ریفکتور

```
csharp

public class OrderCalculator
{
    public decimal CalculateOrderPrice(Order order, Customer customer, DateTime orderDate)
    {
        // محاسبه قیمت پایه
        decimal basePrice = 0;
        foreach (var item in order.Items)
        {
            decimal itemPrice = item.Price * item.Quantity;

            if (item.IsOnSale)
                itemPrice *= 0.9m;

            if (item.Quantity > 10)
                itemPrice *= 0.95m;

            basePrice += itemPrice;
        }

        // محاسبه تخفیف مشتری
        decimal customerDiscount = 0;
        if (customer.IsVIP)
        {
```

```

        if (customer.YearsAsMember > 5)
            customerDiscount = basePrice * 0.15m;
        else
            customerDiscount = basePrice * 0.1m;
    }
    else if (customer.YearsAsMember > 3)
    {
        customerDiscount = basePrice * 0.05m;
    }
}

// محاسبه تخفیف فصلی
decimal seasonalDiscount = 0;
if (orderDate.Month == 3 || orderDate.Month == 12)
{
    seasonalDiscount = basePrice * 0.08m;
}
else if (orderDate.Month >= 6 && orderDate.Month <= 8)
{
    seasonalDiscount = basePrice * 0.05m;
}

// محاسبه مالیات
decimal taxRate = 0.09m;
if (customer.City == "Tehran")
    taxRate = 0.1m;
decimal tax = (basePrice - customerDiscount - seasonalDiscount) * taxRate;

// محاسبه هزینه حمل
decimal shippingCost = 0;
if (basePrice > 500000)
    shippingCost = 0; // رایگان
else if (basePrice > 200000)
    shippingCost = 15000;
else
    shippingCost = 30000;

// اضافه کردن هزینه بسته‌بندی
if (order.NeedsGiftWrapping)
    shippingCost += 10000;

// قیمت نهایی
decimal finalPrice = basePrice - customerDiscount - seasonalDiscount + tax + shippingCost;

// اعمال حداقل قیمت
if (finalPrice < 50000)
    finalPrice = 50000;

return finalPrice;
}
}

```

این متد با ۵ بخش مختلف، کلی متغیر محلی که به هم وابسته‌ان، و منطق پیچیده — استخراج متد معمولی اینجا ساخته!

✅ Replace Method with Method Object

csharp

```
// ۱: ساختن کلاس جدید برای مند
public class OrderPriceCalculator
{
    // متغیرهای محلی سابق -> فیلدهای کلاس
    private readonly Order _order;
    private readonly Customer _customer;
    private readonly DateTime _orderDate;

    private decimal _basePrice;
    private decimal _customerDiscount;
    private decimal _seasonalDiscount;
    private decimal _tax;
    private decimal _shippingCost;

    public OrderPriceCalculator(Order order, Customer customer, DateTime orderDate)
    {
        _order = order;
        _customer = customer;
        _orderDate = orderDate;
    }

    // ۲: مند اصلی که حالا قابل شکستن
    public decimal Calculate()
    {
        _basePrice = CalculateBasePrice();
        _customerDiscount = CalculateCustomerDiscount();
        _seasonalDiscount = CalculateSeasonalDiscount();
        _tax = CalculateTax();
        _shippingCost = CalculateShippingCost();

        return ApplyMinimumPrice(ComputeFinalPrice());
    }

    // ۳: هر بخش به یک متد کوچک تبدیل شده
    private decimal CalculateBasePrice()
    {
        decimal basePrice = 0;
        foreach (var item in _order.Items)
        {
            decimal itemPrice = item.Price * item.Quantity;

            if (item.IsOnSale)
                itemPrice *= 0.9m;

            if (item.Quantity > 10)
                itemPrice *= 0.95m;

            basePrice += itemPrice;
        }
        return basePrice;
    }

    private decimal CalculateCustomerDiscount()
    {
        if (_customer.IsVIP)
        {
            if (_customer.YearsAsMember > 5)
```

```

        return _basePrice * 0.15m;
    else
        return _basePrice * 0.1m;
    }
    else if (_customer.YearsAsMember > 3)
    {
        return _basePrice * 0.05m;
    }
    return 0;
}

private decimal CalculateSeasonalDiscount()
{
    if (_orderDate.Month == 3 || _orderDate.Month == 12)
        return _basePrice * 0.08m;

    if (_orderDate.Month >= 6 && _orderDate.Month <= 8)
        return _basePrice * 0.05m;

    return 0;
}

private decimal CalculateTax()
{
    decimal taxRate = _customer.City == "Tehran" ? 0.1m : 0.09m;
    return (_basePrice - _customerDiscount - _seasonalDiscount) * taxRate;
}

private decimal CalculateShippingCost()
{
    decimal cost = _basePrice > 500000 ? 0 :
        _basePrice > 200000 ? 15000 : 30000;

    if (_order.NeedsGiftWrapping)
        cost += 10000;

    return cost;
}

private decimal ComputeFinalPrice()
{
    return _basePrice - _customerDiscount - _seasonalDiscount + _tax + _shippingCost;
}

private decimal ApplyMinimumPrice(decimal price)
{
    return price < 50000 ? 50000 : price;
}
}

// قدم ۴: کلاس اصلی فقط صدا میزنه
public class OrderCalculator
{
    public decimal CalculateOrderPrice(Order order, Customer customer, DateTime orderDate)
    {
        return new OrderPriceCalculator(order, customer, orderDate).Calculate();
    }
}

```

```
}  
}
```

مزایا

مزیت	توضیح
متد اصلی خیلی ساده	فقط صدا زدن کلاس جدید
متغیرهای محلی حذف شدن	تبدیل شدن به فیلد — بدون رد و بدل بین متدها
قابل شکستن	کنی Extract Method حالا میتونی راحت
تست مجزا	هر متد کوچیک رو میتونی جدا تست کنی
Single Responsibility	کلاس جدید فقط یه کار داره: محاسبه قیمت

تمرین برای تو

ریفکتور کن **Replace Method with Method Object** این کد رو با

```
csharp  
  
public class ReportBuilder  
{  
    public string BuildSalesReport(List<Sale> sales, int year, string region,  
                                   bool includeDetails, bool includeCharts, string forma  
t)  
    {  
        // فیلتر کردن فروش‌ها  
        var filteredSales = new List<Sale>();  
        foreach (var sale in sales)  
        {  
            if (sale.Date.Year == year && sale.Region == region)  
                filteredSales.Add(sale);  
        }  
  
        // محاسبه آمار  
        decimal totalAmount = 0;  
        int totalCount = filteredSales.Count;  
        decimal maxSale = 0;  
        decimal minSale = decimal.MaxValue;  
        var monthlyTotals = new Dictionary<int, decimal>();  
  
        foreach (var sale in filteredSales)  
        {  
            totalAmount += sale.Amount;  
  
            if (sale.Amount > maxSale)  
                maxSale = sale.Amount;  
            if (sale.Amount < minSale)  
                minSale = sale.Amount;  
        }  
    }  
}
```

```

        int month = sale.Date.Month;
        if (!monthlyTotals.ContainsKey(month))
            monthlyTotals[month] = 0;
        monthlyTotals[month] += sale.Amount;
    }

    decimal average = totalCount > 0 ? totalAmount / totalCount : 0;

    // ساختن هدر گزارش
    string report = $"گزارش فروش - {year} - منطقه {region}\n";
    report += $"تعداد فروش: {totalCount}\n";
    report += $"فروش کل: {totalAmount:N0} یل\n";
    report += $"میانگین: {average:N0} یل\n";
    report += $"حداکثر: {maxSale:N0} یل\n";
    report += $"حداقل: {(minSale == decimal.MaxValue ? 0 : minSale):N0} یل\n";

    // اضافه کردن جزئیات ماهانه
    report += "\nفروش ماهانه:\n";
    for (int i = 1; i <= 12; i++)
    {
        if (monthlyTotals.ContainsKey(i))
            report += $"ماه {i}: {monthlyTotals[i]:N0} یل\n";
    }

    // اضافه کردن جزئیات اگر خواسته شده باشد
    if (includeDetails)
    {
        report += "\nجزئیات فروش ماهانه:\n";
        foreach (var sale in filteredSales)
        {
            report += $"- {sale.Date:yyyy/MM/dd}: {sale.Product} - {sale.Amount:N0} یل\n";
        }
    }

    // اضافه کردن نمودار اگر خواسته شده باشد
    if (includeCharts)
    {
        report += "\nنمودار فروش ماهانه:\n";
        for (int i = 1; i <= 12; i++)
        {
            if (monthlyTotals.ContainsKey(i))
            {
                int barLength = (int)(monthlyTotals[i] / 10000);
                report += $"ماه {i:D2}: {new string('█', barLength)}\n";
            }
        }
    }

    // فرمت‌بندی نهایی
    if (format == "HTML")
    {
        report = report.Replace("\n", "<br>");
        report = $"<html><body>{report}</body></html>";
    }

    return report;
}
}

```

```
public class Sale
{
    public DateTime Date { get; set; }
    public string Region { get; set; }
    public string Product { get; set; }
    public decimal Amount { get; set; }
}
```

راهنمایی:

- به کلاس SalesReportBuilder بساز
- پارامترهای متد + filteredSales و بقیه متغیرهای محلی → فیلدهای کلاس
- بشه متد Build() که از چند متد کوچک تشکیل شده BuildSalesReport
- هر بخش رو ببر به متد جدا: FilterSales , CalculateStatistics , BuildHeader , BuildMonthlySection , BuildDetails , BuildCharts , FormatReport

👉!بفرست تا بررسی کنیم